Einführung

Parameter	Kursinformationen
Veranstaltung:	`Robotik Projekt`
Semester	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Einführung in die mobile Robotik`
Link auf GitHub:	https://github.com/TUBAF-IfI-LiaScript/VL_SoftwareprojektRobotik/blob/master/00_Einfuehrung/00_Einfuehrung.md
Autoren	Sebastian Zug & Georg Jäger & Copilot & Claude Code (Opus 5.5) mit LiaScript Teaching-Agent



Zielstellung der heutigen Veranstaltung

Nach dieser Vorlesung können Sie

- die Projektaufgabe und den Ablauf der Veranstaltung (VL, Ü, P, Prüfungsleistung) beschreiben,
- entscheiden, ob ein System nach unserer Arbeitsdefinition ein Roboter ist, und es nach Autonomie, Mobilität und Sensorik einordnen,
- für ein Einsatzszenario die wichtigsten Herausforderungen benennen,
- Aufgaben des TurtleBots den Schritten Sense, Think und Act zuordnen,
- begründen, warum wir ein Framework wie ROS 2 einsetzen.

Wo stehen wir?

Kurz zur Orientierung. Wir sind in Block 1, Termin 1 von 32. Sie lernen die Projektaufgabe kennen: Ein TurtleBot soll autonom durch die Verkehrswelt fahren. Neu sind heute: Projektaufgabe Verkehrswelt, Roboter, Sense-Think-Act, TurtleBot 3 Burger, Warum ROS?.

Ziel im Projekt: Sie lernen die Projektaufgabe kennen: Ein TurtleBot soll autonom durch die Verkehrswelt fahren.

Block 1 · ROS-Grundlagen – Termin 1 von 32, heute farbig markiert

Woche ab	Vorlesung	Übung	Praktikum
19.10.2026	**VL 1 Einführung**		
26.10.2026	VL 2 ROS 2 Grundlagen	Ü 1 Linux Shell	
2.11.2026	VL 3 ROS 2 Pakete		
9.11.2026	VL 4 Koordinaten + QoS	Ü 2 ROS 2 Werkzeuge	
23.11.2026			P 1 TurtleBot + Bags

- **Sie brauchen:** nichts – wir fangen an
- **Heute neu:** Projektaufgabe Verkehrswelt · Roboter · Sense-Think-Act · TurtleBot 3 Burger · Warum ROS?

> Tip
>
> **Tipp – Kurslandkarte:** Ganze Kurslandkarte öffnen – dort finden Sie jeden Begriff mit Erklärung, typischer Stolperfalle, Link auf die passende Folie und die roten Fäden durch die Veranstaltung.

Organisation

> **Tip**
>
> Die Veranstaltung beginnt im Wintersemester und setzt sich im Sommersemester fort. Für einige Studiengänge gibt es aber auch eine kleine Variante, die nur im Sommersemester läuft.

Anders als in den vergangenen Jahren trennen wir Vorlesung und Projekt nicht mehr auf zwei Semester. Vorlesung, Übung und Praktikum greifen von Anfang an ineinander. Jeder Termin dauert 90 Minuten. In manchen Wochen findet keine Vorlesung statt; diese Zeit gehört dann ganz der Arbeit am Roboter.

Format	Wo?	Was passiert?
Vorlesung (VL)	Hörsaal	Konzepte und Methoden, Beispiele aus realen Robotersystemen
Übung (Ü)	eigener Rechner / Pool	angeleitete Aufgaben, z. B. mit aufgezeichneten Sensordaten (Bags)
Praktikum (P)	Labor, TurtleBot	Gruppenarbeit (2–3 Personen pro Roboter) an der Projektaufgabe

https://tubaf-ifi-liascript.github.io/VL_SoftwareprojektRobotik/kurslandkarte/#termin=00

Mit der Veranstaltung erlangen Sie nominell 9 ECTS Punkte. Das bedeutet, dass Sie insgesamt 270 Stunden Arbeit investieren müssen. Die Vorlesung ist nur ein Teil davon: Sie vermittelt die Konzepte, die Sie in den Übungen und im Praktikum umsetzen. Die Prüfungsleistung erbringen Sie in den Praktika.

Für die Arbeit benötigen Sie ein Linux-System. Wir arbeiten mit Ubuntu 24.04 und ROS 2 Jazzy. Wie Sie das auf Ihrem Rechner einrichten, klären wir in den ersten Übungen.

Werkzeuge

- Ubuntu 24.04 (nativ, alternativ WSL2 unter Windows)
- ROS 2 Jazzy
- Python und C++
- Git / GitHub für die Gruppenarbeit

Organisatorisches

Thema	Regelung
Prüfungsleistung	Ihre Arbeit in den **Praktika**: Dort wenden Sie das Gelernte am TurtleBot an und bauen Schritt für Schritt die Projektaufgabe auf.
Vorlesung, Übung	vermitteln und üben, was Sie im Praktikum brauchen — die Selbsttests zeigen Ihnen, ob Sie bereit sind
Gruppen	2–3 Personen pro TurtleBot, gebildet bis zum ersten Praktikum P 1
Versionsverwaltung	jede Gruppe arbeitet in einem eigenen Git-Repository

Wo stehen Sie?

Wichtig für Ihre Planung: Die Prüfungsleistung erbringen Sie in den Praktika. Vorlesung und Übung bereiten Sie darauf vor — wer die Übungen auslässt, steht im Praktikum vor dem Roboter und weiß nicht weiter.

Wo stehen Sie? Ihre Antworten helfen uns, die Übungen und die Gruppen passend zu planen.

Wie sicher sind Sie im Umgang mit dem Linux-Terminal?

- ◯ sehr sicher
- ◯ grundlegende Erfahrung
- ◯ kaum / keine Erfahrung

Mit welchen Themen haben Sie schon gearbeitet?

- ☐ Python
- ☐ C++
- ☐ Git
- ☐ ROS / ROS 2
- ☐ OpenCV / Bildverarbeitung
- ☐ Mikrocontroller / Embedded

Unser Ziel: Autonomes Fahren im Miniaturformat

Bevor wir über Definitionen sprechen, schauen wir auf das Ziel dieser Veranstaltung. Am Ende des Sommersemesters soll Ihr TurtleBot selbstständig durch eine kleine Verkehrswelt fahren: Er folgt der Straße, hält an der roten Ampel, beachtet Verkehrszeichen, lässt Fußgänger passieren und parkt am Ende ein. Alles, was Sie in Vorlesung und Übung lernen, ist ein Baustein für dieses Gesamtsystem.



TurtleBot 3 im Praktikum: Fahrspur, Parkbuchten und Verkehrszeichen

Die Projektaufgabe

Ein TurtleBot 3 Burger bewegt sich autonom in einer Miniatur-Verkehrswelt auf schwarzem Untergrund und muss dabei

- **Straßen und Fahrspuren** erkennen und ihnen folgen,
- **Ampelanlagen** und **Verkehrszeichen** erkennen und befolgen,
- **Fußgängern** und Hindernissen ausweichen bzw. vor ihnen anhalten,
- **einparken** und weitere Fahrmanöver ausführen.

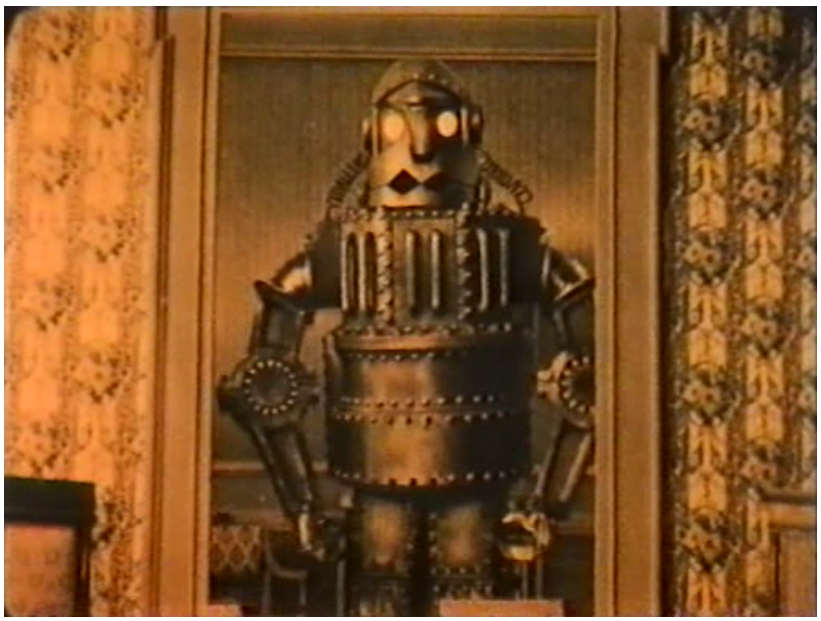
Diese Aufgabe lösen Sie nicht an einem Stück, sondern Block für Block. Jeder Block der Veranstaltung liefert eine Teilfähigkeit, die Sie im Praktikum direkt auf dem Roboter umsetzen.

Block	Thema	Beitrag zur Projektaufgabe	Semester
1	ROS Einführung und Grundlagen	Roboter ansteuern, Daten aufzeichnen und auswerten	WiSe
2	Sensorgrundlagen	Kamerabild korrigieren, Straße/Fahrspur erkennen	WiSe
3	Objekterkennung	Ampeln, Schilder, Fußgänger und Hindernisse erkennen	WiSe
4	Sensorfusion	Position und Umgebung robust schätzen	SoSe
5	Regelung und Navigation	Bahnfolge, Karte, Einparken	SoSe
6	Praktikum + X	Integration zum Gesamtsystem	SoSe

Was ist ein Roboter?

Bevor wir Definitionen lesen: Was halten Sie selbst für einen Roboter? Stimmen Sie ab — Sie werden sehen, dass wir uns im Saal nicht einig sind. Genau so geht es auch den Fachverbänden.

Roboter oder nicht? Welches bild haben Sie im Kopf, wenn Sie an einen Roboter denken?



Screenshot aus dem Film Mechanical Man von 1921

Definition nach VDI-Richtlinie 2860: „Industrieroboter sind universell einsetzbare Bewegungsautomaten mit mehreren Achsen, deren Bewegungen hinsichtlich Bewegungsfolge und Wegen bzw. Winkeln frei (d. h. ohne mechanischen bzw. menschlichen Eingriff) programmierbar und gegebenenfalls sensorgeführt sind. Sie sind mit Greifern, Werkzeugen oder anderen Fertigungsmitteln ausrüstbar und können Handhabungs- und/oder Fertigungsaufgaben ausführen."

Definition nach Robotic Industries Association (RIA): „A robot is a reprogrammable, multifunctional manipulator designed to move material, parts, tools or specialized devices through variable programmed motions for the performance of a variety of tasks"

Klassifikation der Japan Robot Association (JRA) anzeigen ›

Beide Definitionen stammen aus der Industrie und beschreiben Roboterarme. Für mobile Roboter wie den TurtleBot passen sie schlecht.

Für diese Veranstaltung arbeiten wir deshalb mit einer einfachen, funktionalen Definition. Sie hat den Vorteil, dass sie direkt zum Aufbau eines Roboters führt — darum geht es gleich.

Important

Merke – Arbeitsdefinition: Ein Roboter ist eine Maschine, die ihre Umgebung über **Sensoren wahrnimmt**, daraus **Entscheidungen ableitet** und über **Aktoren auf die Umgebung einwirkt** — und das wiederholt, ohne dass ein Mensch jeden Schritt vorgibt.

Der Geschirrspüler misst zwar Wassertemperatur, entscheidet aber nichts über seine Umgebung; die ferngesteuerte Drohne nimmt wahr und handelt, aber ein Mensch entscheidet. Saugroboter, Spurhalteassistent und TurtleBot erfüllen die Definition.

Klassifikation von Robotersystemen

Roboter lassen sich anhand verschiedener Kriterien klassifizieren. Diese Unterscheidungsmerkmale helfen uns, die Vielfalt der Robotersysteme zu verstehen und ihre spezifischen Anforderungen zu erkennen. In dieser Veranstaltung konzentrieren wir uns vor allem auf autonome, mobile Roboter mit komplexer Sensorik.

Kriterium	Optionen
Art der Steuerung	autonom, teleoperiert, hybrid
Bewegungsfähigkeit	stationär, mobil
Anwendungsbereich	Industrie, Verkehr, Medizin …
Erscheinung	Humanoid, Nicht-humanoid
Energieversorgung	Autark, Batterien, Kabelgebunden
Interaktionsfähigkeit	Kooperativ, Isoliert
Komplexität der Umgebung	Niedrig, Hoch
Sensorik und Wahrnehmung	Einfach, Komplex
Größe	Mikroroboter, Makroroboter
…	…

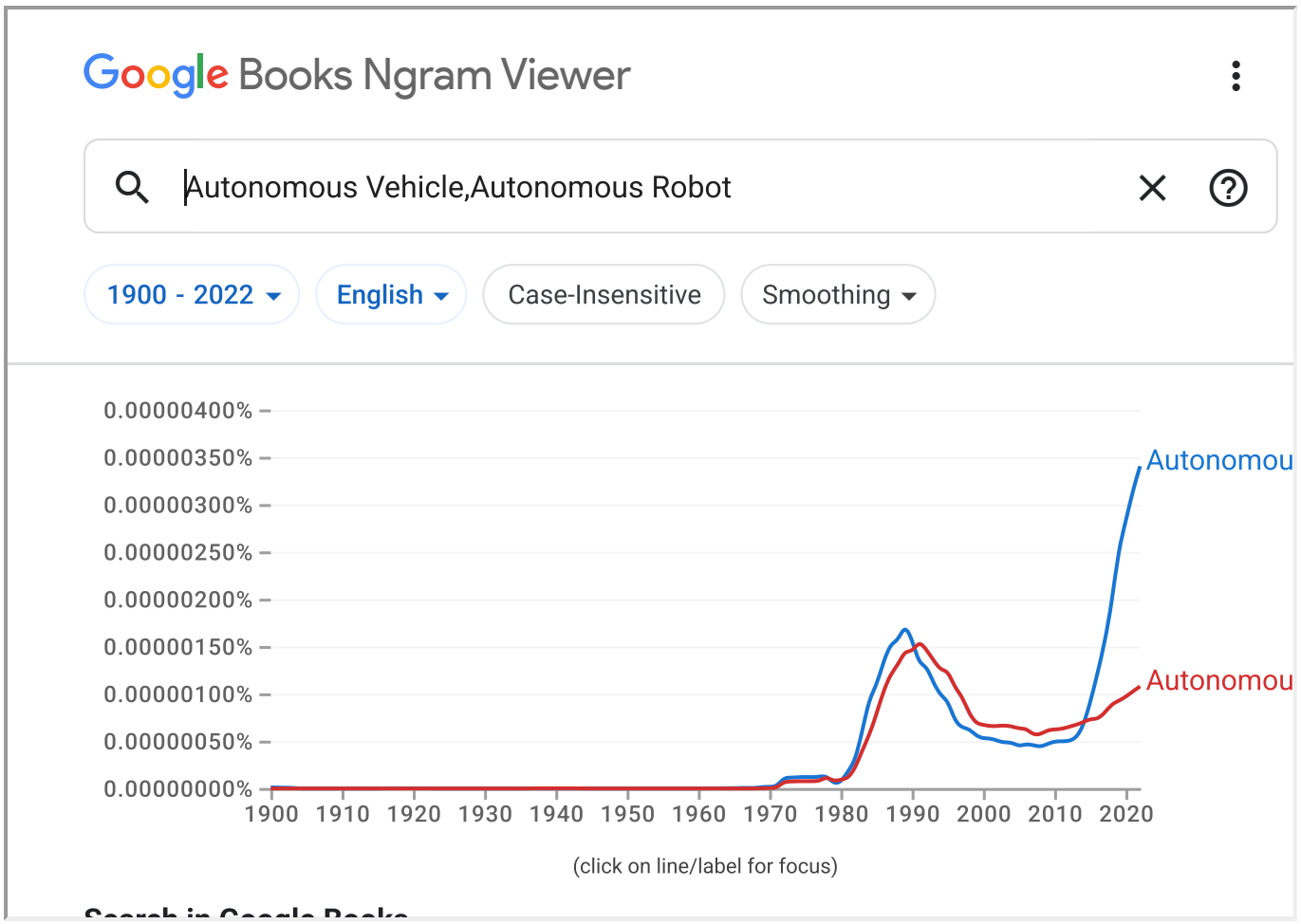
In dieser Veranstaltung konzentrieren wir uns primär auf autonome, mobile Roboter, die in komplexen Umgebungen agieren müssen. Diese Systeme stellen besondere Anforderungen an Sensorik, Datenverarbeitung und Entscheidungsfindung.

Selbsttest: Ordnen Sie die Systeme zu. Welche Eigenschaften treffen zu?

autonom	mobil	komplexe Sensorik	
☐	☐	☐	Mähroboter, der sich an einem Begrenzungsdraht orientiert
☐	☐	☐	Schweißroboter an einer Fertigungsstraße mit festem Programm
☐	☐	☐	ferngesteuerter Inspektionsroboter mit Kamera und LiDAR
☐	☐	☐	unser TurtleBot in der Verkehrswelt

Relevanz und Bedeutung mobiler Roboter

Die Bedeutung mobiler Roboter hat in den letzten Jahrzehnten massiv zugenommen. Dies lässt sich anhand der Häufigkeit von Begriffen in wissenschaftlichen Publikationen nachvollziehen. Die folgende Grafik zeigt die Entwicklung der Begriffe „Autonomous Vehicle“ und „Autonomous Robot“.



Ngram Analyse der Begriffe Autonomous Vehicle und Autonomous Robot

Besonders auffällig ist der exponentielle Anstieg ab den 2010er Jahren, der mit Fortschritten im Machine Learning und Deep Learning zusammenfällt. Diese Technologien sind heute unverzichtbare Werkzeuge für autonome Robotersysteme.

Herausforderungen bei der Entwicklung mobiler Roboter

Die Entwicklung autonomer mobiler Roboter ist eine hochkomplexe Aufgabe, die Herausforderungen auf mehreren Ebenen mit sich bringt. Schauen wir uns diese systematisch an.

Welche technologischen Herausforderungen müssen wir bei mobilen Robotersystemen meistern?

Auf der technologischen Ebene stehen wir vor fundamentalen Problemen: Wie kann ein Roboter seine Position präzise bestimmen? Wie erstellt er eine Karte seiner Umgebung? Wie erkennt und umfährt er Hindernisse? All diese Fragen werden wir im Verlauf dieser Veranstaltung adressieren.

Technologische Herausforderungen

- Robuste, hinreichend präzise **Positionierung**
- **Umgebungskartierung** (SLAM - Simultaneous Localization and Mapping)
- **Hindernisidentifikation** und -umgehung
- **Echtzeit-Umsetzung** von Teilverhalten
- **Energieeffizienz** für längere Betriebszeiten
- Umgang mit **veränderlichen Kommunikationsbedingungen**

Neben der technischen Realisierbarkeit müssen Robotersysteme auch wirtschaftlich sinnvoll sein. Die Entwicklungskosten müssen sich durch den Nutzen rechtfertigen lassen, und die Systeme müssen marktreif sein.

Wirtschaftliche Herausforderungen

- **Wirtschaftlichkeit**: Kosten-Nutzen-Verhältnis
- **Marktreife**: Zuverlässigkeit für den produktiven Einsatz
- **Skalierbarkeit**: Vom Prototyp zur Serie

Schließlich dürfen wir die gesellschaftlichen Aspekte nicht vergessen. Autonome Systeme müssen sicher sein, rechtlichen Vorgaben entsprechen und von der Gesellschaft akzeptiert werden. Ethische Fragen wie der Umgang mit Datenschutz oder potenzielle Arbeitsplatzverdrängung sind hochrelevant.

Soziale und rechtliche Herausforderungen

- **Sicherheitsanforderungen**: Schutz von Menschen und Infrastruktur
- **Regulatorische Rahmenbedingungen**: Gesetze und Normen
- **Ethik und Datenschutz**: Verantwortungsvoller Umgang mit Daten
- **Gesellschaftliche Akzeptanz**: Arbeitsplatzverdrängung, Vertrauen

Betrachten wir zwei konkrete Beispiele aus der Praxis: einen Lieferroboter für städtische Umgebungen und einen aquatischen Roboter für Gewässerüberwachung. Welche der genannten Herausforderungen sind jeweils besonders relevant?

Diskussion: Welche dieser Herausforderungen sehen Sie als besonders relevant für folgende Szenarien?

1. **Lieferroboter** in einer Fußgängerzone
2. **Aquatischer Roboter** zur Gewässerüberwachung



Autonomer Roboter des Ready for Robots Projektes



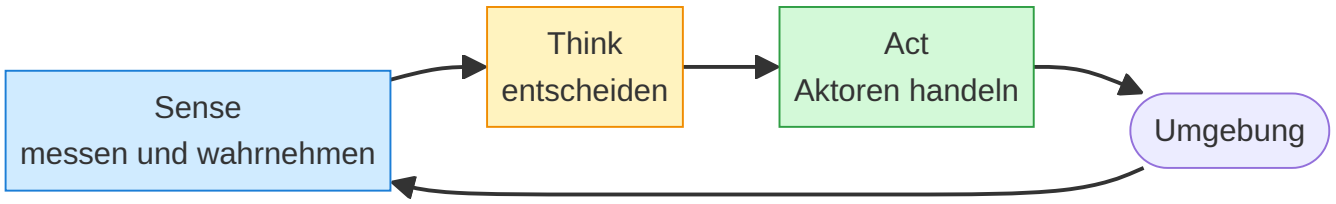
TUBAF Schwimmroboter mit Windmessungsaufsatz

Diskutieren Sie 3 Minuten mit Ihrem Nachbarn und notieren Sie für jedes Szenario die zwei wichtigsten Herausforderungen.

Mögliche Antworten anzeigen ›

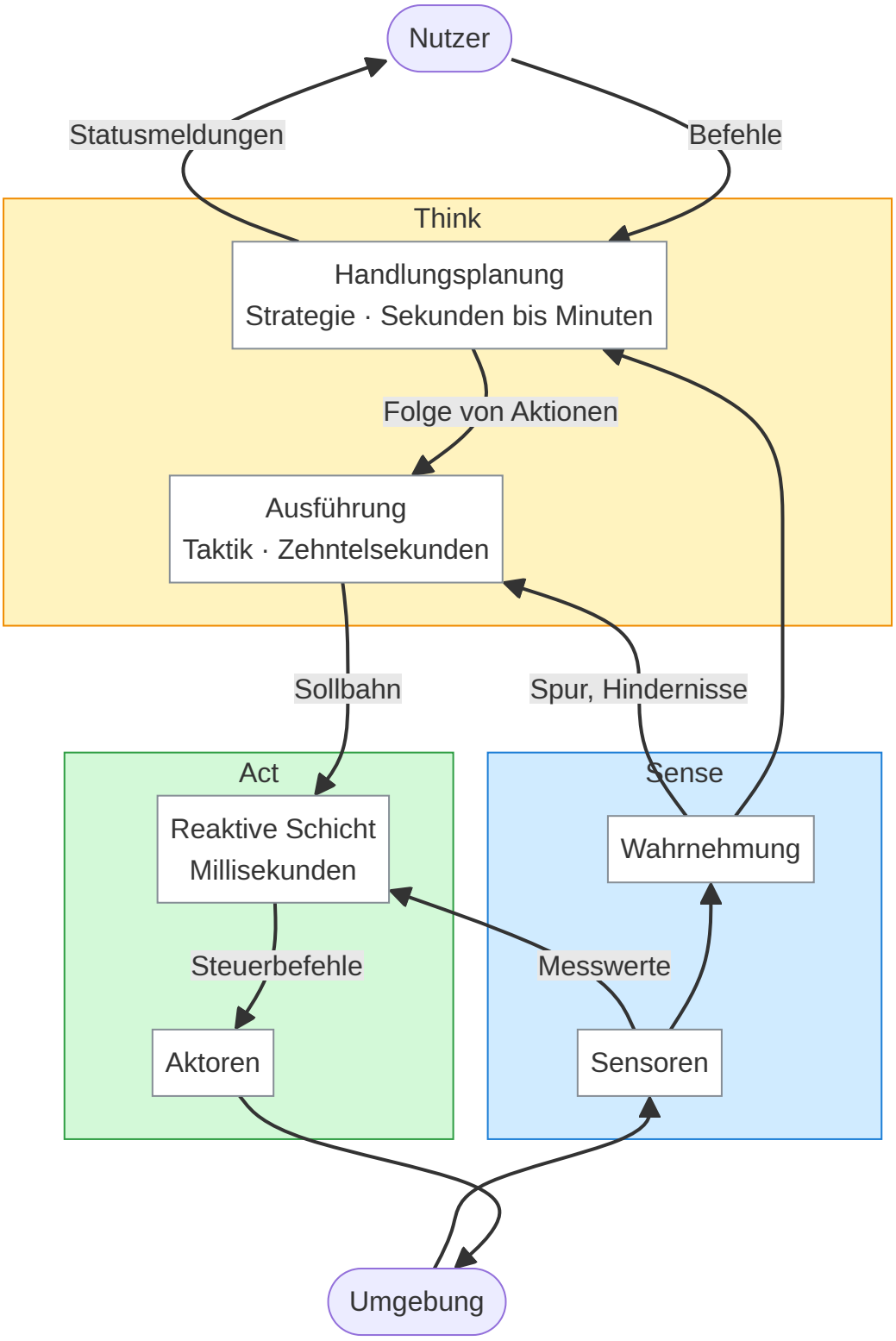
Systemarchitektur: Das Sense-Think-Act-Paradigma

Unsere Arbeitsdefinition beschreibt bereits den Aufbau jedes Roboters: wahrnehmen, entscheiden, handeln. Diesen Kreislauf nennt man das Sense-Think-Act-Paradigma. Er läuft ständig: Jede Handlung verändert die Umgebung, und die Sensoren nehmen die Veränderung im nächsten Durchlauf wieder wahr.



Streng nacheinander ausgeführt hat der Kreislauf ein Problem: Er ist für schnelle Reaktionen zu langsam. Wenn plötzlich ein Fußgänger vor dem Roboter steht, darf der Notstopp nicht warten, bis ein neuer Plan berechnet ist. Reale Roboter nutzen deshalb eine hybride Architektur mit drei Schichten. Im Bild liegen sie übereinander: oben die langsamste, unten die schnellste. Die beiden oberen bilden das eigentliche Think. Die Handlungsplanung entscheidet in Sekunden bis Minuten, welches Ziel als Nächstes angesteuert wird, die Ausführung setzt das in Zehntelsekunden in Fahrmanöver um. Die unterste, die reaktive Schicht, ordnen wir Act zu. Sie bekommt die Messwerte direkt von den Sensoren und reagiert in Millisekunden, ohne zu überlegen. Warum sie trotzdem nicht zu Think gehört, obwohl sie den Notstopp auslöst, steht im Hinweis unter dem Bild.

Schnell und überlegt: die hybride Drei-Schichten-Architektur



Sense (blau), Think (gelb) und Act (grün) sind dieselben Schritte wie im Kreislauf oben. Von den Sensoren führen zwei Wege zu den Aktoren:

- der **überlegte** über Wahrnehmung, Handlungsplanung und Ausführung,
- der **schnelle, direkte** über die reaktive Schicht.

Am Beispiel Kurve: Die Ausführung entscheidet „jetzt der Spur folgen“ und gibt die *Sollbahn* vor, also den Weg, den der Roboter fahren soll. Die reaktive Schicht regelt die Räder alle paar Millisekunden so nach, dass er auf dieser Bahn bleibt.

> **Caution**
>
> **Auch die reaktive Schicht entscheidet.** Der Notstopp wählt zwischen „weiterfahren“ und „stoppen“ — warum steht er dann bei Act und nicht bei Think? Weil er nur einer festen Regel folgt („Abstand unter 15 cm → stoppen“) und keine Alternativen abwägt. „Bei Gelb fahren oder halten?“ wägt dagegen ab, etwa Abstand, Geschwindigkeit und Verkehrslage, und gehört deshalb zu Think. Die Zuordnung ist eine Vereinfachung für diesen Kurs: In der Literatur wird die reaktive Schicht oft auch der Steuerung, also Think, zugerechnet.

Die Tabelle zeigt, welche Aufgaben auf den einzelnen Schichten anfallen — von Hardwaretreibern über Regelung und Weltmodell bis zur Verhaltensplanung. Und genau hier stellt sich die entscheidende Frage: Wer soll das alles implementieren?

Schicht	Typische Aufgaben
Handlungsplanung	Weltmodell und Karte, Routen- und Verhaltensplanung, Nutzerschnittstelle
Ausführung	Ablaufsteuerung, Handlungsmuster („Spur folgen“, „einparken“), passendes Manöver wählen
Reaktive Schicht	Regelung auf die Sollbahn, Notstopp, Überwachung von Akku und Motoren
Sensoren, Wahrnehmung, Aktoren (keine Schicht)	Hardwaretreiber; Wahrnehmung, z.B. Spur im Bild erkennen

Zentrale Frage: Wer soll das denn alles implementieren?

Sense-Think-Act am TurtleBot

Übertragen wir das Schema auf unseren eigenen Roboter. Der TurtleBot 3 Burger ist klein, aber er bringt alle drei Schritte mit: Sensoren zur Wahrnehmung, einen Rechner für die Entscheidungen und Motoren für die Bewegung.

Komponente	Spezifikation
LiDAR	RPLIDAR C1 (360°, 12 m Reichweite, 10 Hz)
Kamera	Raspberry Pi Camera v2
Antrieb	Differentialantrieb (DYNAMIXEL)
Max. Geschwindigkeit	0,22 m/s
Rechner	Raspberry Pi 3 Model B+

Selbsttest 1: Zu welchem Schritt des Kreislaufs gehört die jeweilige Aufgabe in unserer Verkehrswelt?

Sense	Think	Act	
◯	◯	◯	Entscheiden, ob bei Gelb noch gefahren wird
◯	◯	◯	Kamerabild der Fahrbahn erfassen
◯	◯	◯	Die Spur im Kamerabild erkennen
◯	◯	◯	Radgeschwindigkeiten über die Motoren einstellen
◯	◯	◯	Abstand zu einem Fußgänger mit dem LiDAR messen
◯	◯	◯	Einen Pfad in die Parklücke planen

Selbsttest 2: Welche Schicht der hybriden Architektur ist zuständig?

Handlungsplanung	Ausführung	Reaktive Schicht	
◯	◯	◯	Entscheiden, als Nächstes die Parklücke anzusteuern
◯	◯	◯	Entscheiden, jetzt der Spur zu folgen statt einzuparken
◯	◯	◯	Die Räder laufend nachregeln, damit der Roboter auf der Sollbahn bleibt
◯	◯	◯	Sofort stoppen, wenn der LiDAR einen Fußgänger direkt vor dem Roboter misst

Warum ROS? Das „Rad nicht neu erfinden“

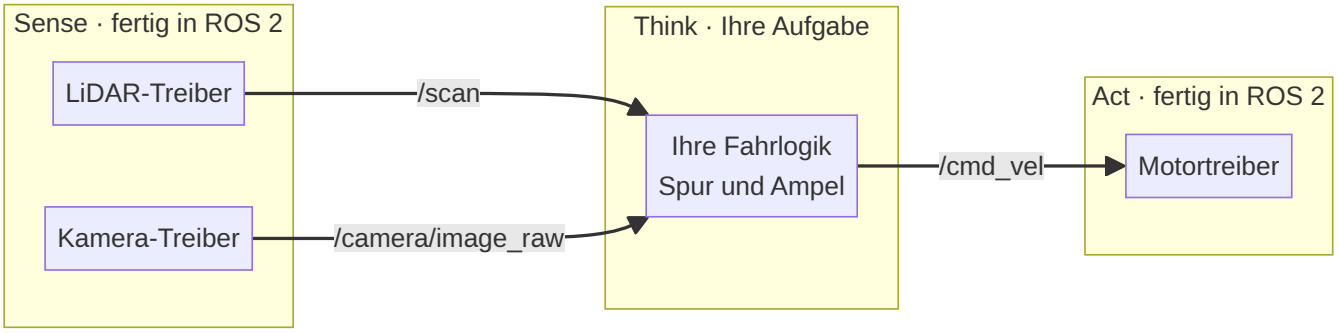
Genau an diesem Punkt kommt das Robot Operating System ins Spiel. ROS ist nicht wirklich ein Betriebssystem im klassischen Sinne, sondern ein Framework, das Standardlösungen für wiederkehrende Robotik-Probleme bereitstellt. Statt alles von Grund auf neu zu entwickeln, können Entwickler auf eine riesige Bibliothek von Werkzeugen und Algorithmen zurückgreifen. Dieser Comic von Willow Garage, dem ursprünglichen Entwickler von ROS, bringt das Problem auf den Punkt: Ohne ein gemeinsames Framework entwickelt jeder sein eigenes System, seine eigenen Treiber, seine eigenen Algorithmen. Das ist ineffizient, fehleranfällig und behindert den wissenschaftlichen Fortschritt.



Willow Garage, http://www.willowgarage.com/blog/2010/04/27/reinventing-wheel, 2010

Übertragen auf unseren TurtleBot heißt das: Die Treiber für LiDAR, Kamera und Motoren bekommen Sie fertig. Ihre Arbeit im Projekt ist der Teil in der Mitte — die Entscheidungen.

So sieht Sense-Think-Act am TurtleBot mit ROS 2 aus. Jeder Kasten ist ein eigenständiges Programm, die Pfeile sind Datenkanäle mit festem Namen:



ROS 2 liefert:

- fertige **Treiber** für Sensoren und Motoren — die Kästen links und rechts,
- einen **einheitlichen Datenaustausch** zwischen allen Programmen,
- **Werkzeuge**, um Daten anzuzeigen, aufzuzeichnen und abzuspielen.

Ihre Arbeit im Projekt ist der Kasten in der Mitte. Wie der Datenaustausch genau funktioniert, ist Thema von VL 2.

Ausblick: Die nächsten Schritte

Bevor wir mit ROS 2 arbeiten können, brauchen wir ein solides Fundament: den sicheren Umgang mit dem Linux-Terminal. Genau darum geht es in der ersten Übung. Danach steigen wir in der Vorlesung in die Konzepte von ROS 2 ein. ROS 2 installieren Sie selbstständig nach einer Anleitung, bevor die zweite Übung beginnt.

Was kommt als Nächstes?

Termin	Inhalt
Ü 1 OS + Linux Shell	Terminal, Dateisystem, Rechte, Prozesse, Umgebungsvariablen
VL 2 Grundlagen ROS 2	Architektur, Knoten, Topics, Nachrichten
VL 3 ROS 2 Pakete	Pakete, Build-System, Launch-Files
Ü 2 Werkzeuge von ROS 2	Services, Parameter, Actions, Daten aufzeichnen (Installation vorab)
VL 4 Koordinaten + QoS	Transformationen, tf2, Quality of Service
P 1 TurtleBot in Betrieb nehmen	erste Fahrt mit dem TurtleBot, Daten aufzeichnen und auswerten

Wenn Sie noch kein Linux-System haben, nutzen Sie die Zeit bis zur Übung. Unter Windows ist das Linux-Subsystem WSL2 eine gute Alternative zu einer nativen Installation.

> Note
>
> **Hinweis – bis zur Übung:** Stellen Sie sicher, dass Sie Zugriff auf ein Ubuntu-24.04-System haben — nativ, als WSL2 unter Windows oder im Rechnerpool.

Zusammenfassung

Fassen wir die wichtigsten Punkte dieser Einführungsvorlesung zusammen:

Was haben wir gelernt?

- Ziel der Veranstaltung ist ein TurtleBot, der **autonom durch eine Miniatur-Verkehrswelt** fährt — aufgebaut Block für Block in VL, Ü und P
- Ein Roboter **nimmt wahr, entscheidet und handelt** — wiederholt und ohne dass ein Mensch jeden Schritt vorgibt (unsere Arbeitsdefinition)
- Robotersysteme lassen sich anhand **vieler Kriterien klassifizieren** (Autonomie, Mobilität, Sensorik, etc.)
- Die Entwicklung mobiler Roboter bringt **technologische, wirtschaftliche und soziale Herausforderungen** mit sich
- Die **Systemarchitektur** folgt dem **Sense-Think-Act-Kreislauf**; reale Roboter ergänzen ihn zu einer **hybriden Architektur** mit drei Schichten: Handlungsplanung und Ausführung (Think) sowie eine reaktive Schicht (Act), die Messwerte direkt in Steuerbefehle umsetzt
- **ROS 2** ermöglicht es uns, **auf vorhandenen Lösungen aufzubauen** statt alles neu zu entwickeln

Weiterführende Ressourcen

Für Interessierte hier noch einige Ressourcen zur Vertiefung:

Literatur und Links:

- ROS 2 Documentation - Offizielle ROS 2 Dokumentation
- The Robotics Back-End - Tutorials und Praxisbeispiele
- RoboCup - RoboCup-Wettbewerbe und -Teams
- TurtleBot 3 e-Manual - Dokumentation des Herstellers ROBOTIS

🗺️ **Nächster Halt:** VL 2 · ROS 2 Grundlagen (Woche ab 26.10.2026) – Sie verstehen, wie die Software des TurtleBots aus Knoten und Topics zusammengesetzt ist. Auf der Kurslandkarte ansehen

Außerdem bis dahin: Ü 1 · Linux Shell (Woche ab 26.10.2026)